arrays. Let's bring back our tweets.json from chapter 2. We can nest and measure it to transform it from an array of tweets into an array of Twitter users with their number of tweets computed:

```
var nestedTweets = d3.nest()
     .key(function (el) {
       return el.user;
     })
     .entries(incData);

nestedTweets.forEach(function (el) {
     el.numTweets = el.values.length;
     el.numFavorites = d3.sum(el.values, function (d) {
          return d.favorites.length;
     });
     el.numRetweets = d3.sum(el.values, function (d) {
          return d.retweets.length;
     });
});
```

Gives the total number of favorites by summing the favorites array length of all the tweets

Gives the total number of retweets by doing the same for the retweets array length

5.2.2 Creating a ring chart

If we try to run `pieChart(nestedTweets)` like with the earlier array illustrated in figure 5.4, it will fail, because it doesn't know that the numbers we should be using to size our pie pieces come from the `.numTweets` attribute. Most layouts, pie included, can define where the values are in your array by defining an accessor function to get to those values. In the case of `nestedTweets`, we define `pieChart.value()` to point at the `numTweets` attribute of the dataset it's being used on. While we're at it, let's set a value for our arc generator's `innerRadius()` so that we create a donut chart instead of a pie chart. With those changes in place, we can use the same code as before to draw the pie chart in figure 5.6:

```
pieChart.value(function(d) {
                  return d.numTweets;
               });
newArc.innerRadius(20)
yourPie = pieChart(nestedTweets);
```

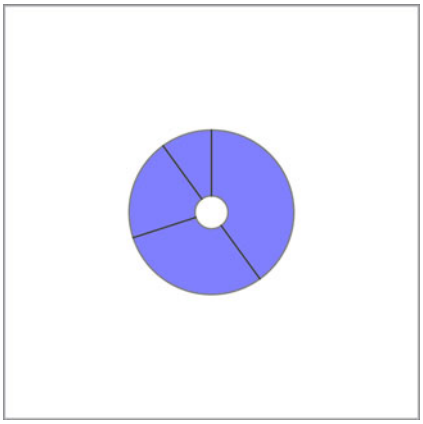


Figure 5.6 A donut chart showing the number of tweets from our four users represented in the `nestedTweets` dataset